

React Lifecycle Methods & useEffect Hook

Every React component has a **life cycle**, just like humans — it is created, it updates, and it is destroyed. React gives us special methods (in class components) and hooks (in functional components) to control what happens in each stage.

Mounting Phase

This is when the component is first created and added to the screen. We use `componentDidMount()` in class components or `useEffect(() => {}, [ ])` in functional components to do tasks like calling an API or setting up data.

Updating Phase

This happens when a component's data or props change and React updates the screen. We can use `componentDidUpdate()` in class components or `useEffect(() => {}, [dependencies])` in functional components to do something whenever the data changes.

Unmounting Phase

This is the cleanup phase when the component is removed from the screen. We use `componentWillUnmount()` in class components or cleanup functions in `useEffect` to clear timers or remove listeners.

useEffect Hook (for Functional Components)

The `useEffect()` Hook in React is a special function that lets you perform **side effects** in your functional components.

A **side effect** means any action that happens **outside React's normal rendering**, such as:

- Fetching data from an API
- Working with `localStorage`
- Setting a timer (`setTimeout`, `setInterval`)
- Adding or removing event listeners

`useEffect()` accepts **two arguments**:

- ⇒ **Callback Function** : This is the main function where you write your side effect code.
- ⇒ **Dependency Array** (Optional) : This array tells React **when to run the effect** again.

<u>Dependency</u>	<u>What Happens</u>
<code>[]</code>	Runs only once when the component mounts.
<code>[variable]</code>	Runs every time that variable changes.
<i>(no array)</i>	Runs after every render (not recommended usually).

Common Real-Life Examples of useEffect

- Fetching user data from an API when the dashboard loads.
- Showing current time that updates every second.
- Listening for new chat messages in a messaging app.
- Stopping background music when user leaves the page.
- Saving items in a shopping cart to localStorage whenever cart updates.

Real-World Use Cases in Projects

- In an **e-commerce website**, useEffect is used to load product data when the page opens.
- In a **weather app**, useEffect can fetch live temperature every few minutes.
- In a **fitness tracking app**, it can record and update user steps every second.
- In a **chat app**, it can connect to a socket when the chat starts and disconnect when it ends.
- In a **blog website**, useEffect loads articles and updates them when category changes.

Points To Remember

- Lifecycle means creation, update, and removal of a component.
- useEffect is the functional way to handle lifecycle actions.
- Always clean up inside useEffect to prevent memory leaks.
- Use dependencies correctly to control when useEffect runs.

Important Interview Questions

- What are the three main phases of a React component's lifecycle?
- What is the difference between componentDidMount and componentDidUpdate?
- How does useEffect replace lifecycle methods in functional components?
- What happens if you forget to add dependencies in useEffect?
- What is the purpose of the cleanup function inside useEffect?
- Can useEffect run before the component is visible?
- How do you stop multiple API calls when using useEffect?

Mini Practice Task

Create a small component that shows live time using useEffect(). Update the time every second and stop it when the component unmounts. (Hint: use setInterval and cleanup function.)